МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ

КАРПАТСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ ВАСИЛЯ СТЕФАНИКА

Факультет математики та інформатики

Кафедра комп'ютерних наук та інформаційних систем

ЛАБОРАТОРНА РОБОТА №2

з предмету «Комп'ютерна графіка» (Програмування C++)

Тема: «Побудова фракталів»

Виконав:

студент групи КН-21

Бєлєв Андрій Костянтинович

_____

«____»_____2026 р.

Прийняв:

Никорак Ярослав Ярославович

_____

«____»_____2026 р.

Івано-Франківськ – 2026

1. МЕТА ТА ПОСТАНОВКА ЗАВДАННЯ ЛАБОРАТОРНОЇ РОБОТИ

> **Мета роботи:** вивчити математичні основи фрактальної геометрії, опанувати принципи побудови геометричних самоподібних структур за допомогою рекурсивних алгоритмів мовою C++ та реалізувати візуалізацію базових класів фракталів засобами графічного інтерфейсу Windows GDI.

Постановка завдання: розробити єдиний програмний комплекс з інтерактивним консольним меню та окремим графічним вікном для побудови шести класичних фракталів згідно з методичними вказівками:

- **Завдання 1. Трикутник Серпінського:** реалізувати рекурсивний поділ початкового трикутника шляхом з'єднання середин його сторін до заданого порядку рекурсії n.
- **Завдання 2. Килим Серпінського:** розбивати квадратну область на 9 рівних квадратів, зафарбовуючи центральний та рекурсивно опрацьовуючи 8 периферійних блоків.
- **Завдання 3. Дерево Піфагора (гілки):** побудувати фрактальне бінарне дерево з масштабуванням довжини гілки l на коефіцієнт $0{,}7$ та асиметричними кутами відхилення $+\pi/4$ і $-\pi/6$.
- **Завдання 4. Множина Кантора:** зобразити ітеративне вилучення середньої третини відрізка на послідовних горизонтальних рівнях до глибини n.
- **Завдання 5. Дерево Піфагора (квадрати):** побудувати класичне геометричне дерево Піфагора з квадратів, де сторона кожної наступної пари зменшується у $\sqrt{2}$ разів з поворотом на $\pm\pi/4$.
- **Завдання 6. Крива та сніжинка Коха:** реалізувати генерацію кривої Коха на відрізку та замкненої сніжинки Коха на базі правильного трикутника.

2. МАТЕМАТИЧНІ МОДЕЛІ ТА РЕКУРСИВНІ ЗАЛЕЖНОСТІ

Основою побудови детермінованих геометричних фракталів є заміна початкового геометричного примітива (ініціатора) на самоподібну ламану або сукупність фігур (генератор) на кожному кроці рекурсії. У таблиці 1 наведено математичні співвідношення, покладені в основу розроблених функцій.

Назва фрактала	Функція в коді	Рекурсивні розрахункові співвідношення	Умова зупинки та гілкування
Трикутник Серпінського	`Draw_T1`, `Prov_lin_T1`	Середини сторін: $x_{1n} = (x_1+x_2)/2$, $y_{1n} = (y_1+y_2)/2$ та аналогічно для пар вершин $(2,3)$ і $(3,1)$.	При $n > 0$: по 3 рекурсивні виклики на рівень (3^n трикутників).
Килим Серпінського	`Draw_T2`, `Prov_lin_T2`	Крок сітки $t = (x_2 - x_1)/3$. Центральний квадрат $[x_1+t;\ x_2-t] \times [y_1+t;\ y_2-t]$ зафарбовується.	При $n > 0$: по 8 рекурсивних викликів для сусідніх блоків (8^n).
Дерево Піфагора (гілки)	`Draw_T3`, `lineto1`	Нова довжина $l' = 0{,}7 \cdot l$; кінець гілки: $x' = x + l' \cdot \cos(u)$, $y' = y - l' \cdot \sin(u)$; кути гілок $u + \pi/4$ та $u - \pi/6$.	Триває, доки $l > max_branch$; по 2 виклики з кожної вершини.
Множина Кантора	`Draw_T4`	Довжина третини $t = (x_2 - x_1)/3$. Лівий підвідрізок $[x_1;\ x_1+t]$, правий $[x_2-t;\ x_2]$ зі зміщенням $y + 25$.	При $n > 0$: по 2 рекурсивні виклики (2^n відрізків на рівні n).
Дерево Піфагора (квадрати)	`Draw_T5`, `Rect_T5`	Побудова повернутого квадрата зі стороною l під кутом a. Перехід до сторони $l / \sqrt{2}$ та кутів $a \pm \pi/4$.	Триває при $l > 4{,}0$ пікселів; бінарне розгалуження.
Крива та сніжинка Коха	`Draw_T6`	Точки поділу $A(1/3)$ та $B(2/3)$; вершина зубця C отримується поворотом вектора AB на кут $-\pi/3$.	База рекурсії $n = 0$ (відрізок); при $n > 0$ — 4 виклики (4^n ланок).

3. ОСОБЛИВОСТІ РЕАЛІЗАЦІЇ ГРАФІЧНОЇ ПІДСИСТЕМИ У WINDOWS

4. РЕЗУЛЬТАТИ ВІЗУАЛЬНОГО МОДЕЛЮВАННЯ ФРАКТАЛІВ

На рисунку 1 зображено графічні результати роботи програми у створеному вікні GDI для основних досліджуваних класів фракталів: трикутника Серпінського, килима Серпінського, гілкового дерева Піфагора та сніжинки Коха.

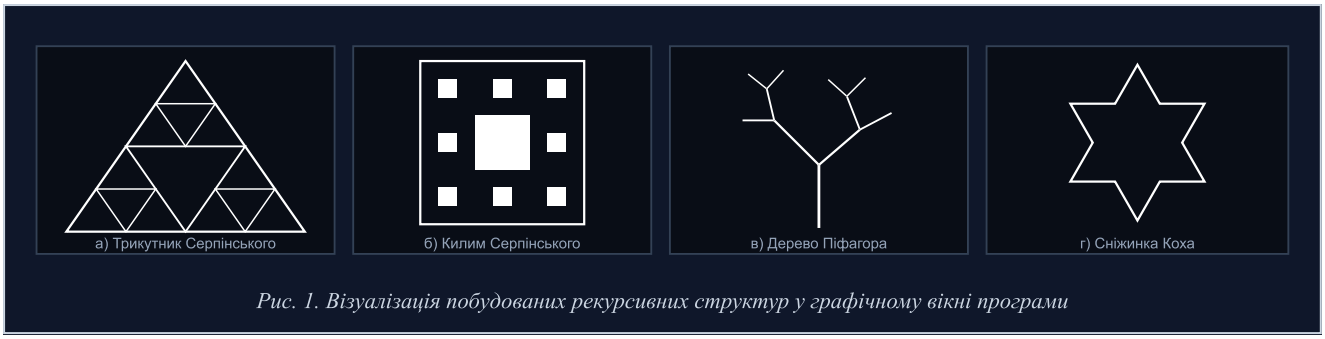


Рис. 1. Візуалізація побудованих рекурсивних структур у графічному вікні програми

5. ПОВНИЙ ЛІСТИНГ ПРОГРАМИ (ФАЙЛ SOURCE.CPP, ЧАСТИНА 1)

Нижче наведено вихідний код ініціалізації графічного вікна Windows GDI, допоміжних примітивів малювання та рекурсивних функцій для перших трьох завдань лабораторної роботи.

```
      #include <windows.h> #include <iostream> #include <cmath> using namespace std; const double PI =
acos(-1.0); // === Графічне вікно Windows GDI === HWND hwndGraph = NULL; HDC hdc = NULL; int curX = 0,
curY = 0; LRESULT CALLBACK WndProc(HWND hWnd, UINT msg, WPARAM wParam, LPARAM lParam) { if (msg ==
WM_DESTROY) { PostQuitMessage(0); return 0; } return DefWindowProc(hWnd, msg, wParam, lParam); } void
InitWinGraph() { HINSTANCE hInstance = GetModuleHandle(NULL); WNDCLASSA wc = { 0 }; wc.lpfnWndProc =
WndProc; wc.hInstance = hInstance; wc.hbrBackground = (HBRUSH)GetStockObject(BLACK_BRUSH);
wc.lpszClassName = "FractalWindowClass"; wc.hCursor = LoadCursor(NULL, IDC_ARROW); RegisterClassA(&wc);
hwndGraph = CreateWindowA("FractalWindowClass", "Лабораторна робота №2: Побудова фракталів",
WS_OVERLAPPEDWINDOW | WS_VISIBLE, 150, 100, 750, 560, NULL, NULL, hInstance, NULL);
ShowWindow(hwndGraph, SW_SHOW); UpdateWindow(hwndGraph); MSG msg; while (PeekMessage(&msg, NULL, 0, 0,
PM_REMOVE)) { TranslateMessage(&msg); DispatchMessage(&msg); } Sleep(100); hdc = GetDC(hwndGraph); RECT
r; GetClientRect(hwndGraph, &r); FillRect(hdc, &r, (HBRUSH)GetStockObject(BLACK_BRUSH));
SelectObject(hdc, GetStockObject(WHITE_PEN)); SelectObject(hdc, GetStockObject(WHITE_BRUSH)); } void
WaitGraphClose() { cout << "\nФрактал побудовано! Закрийте графічне вікно для завершення
програми...\n"; MSG msg; while (GetMessage(&msg, NULL, 0, 0)) { TranslateMessage(&msg);
DispatchMessage(&msg); } } void Line(int x1, int y1, int x2, int y2) { MoveToEx(hdc, x1, y1, NULL);
LineTo(hdc, x2, y2); } void MoveTo(int x, int y) { curX = x; curY = y; MoveToEx(hdc, x, y, NULL); }
void LineToBGI(int x, int y) { MoveToEx(hdc, curX, curY, NULL); LineTo(hdc, x, y); curX = x; curY = y;
} void RectangleFilled(int x1, int y1, int x2, int y2) { RECT r = { x1, y1, x2, y2 }; FillRect(hdc, &r,
(HBRUSH)GetStockObject(WHITE_BRUSH)); } void RectangleOutline(int x1, int y1, int x2, int y2) { HBRUSH
oldBrush = (HBRUSH)SelectObject(hdc, GetStockObject(NULL_BRUSH)); Rectangle(hdc, x1, y1, x2, y2);
SelectObject(hdc, oldBrush); } // ==================== ЗАВДАННЯ 1: ТРИКУТНИК СЕРПІНСЬКОГО
==================== void Prov_lin_T1(double x1, double y1, double x2, double y2, double x3, double y3)
{ Line((int)round(x1), (int)round(y1), (int)round(x2), (int)round(y2)); Line((int)round(x2),
(int)round(y2), (int)round(x3), (int)round(y3)); Line((int)round(x3), (int)round(y3), (int)round(x1),
(int)round(y1)); } void Draw_T1(double x1, double y1, double x2, double y2, double x3, double y3, int
n) { if (n > 0) { double x1n = (x1 + x2) / 2.0, y1n = (y1 + y2) / 2.0; double x2n = (x2 + x3) / 2.0,
y2n = (y2 + y3) / 2.0; double x3n = (x3 + x1) / 2.0, y3n = (y3 + y1) / 2.0; Prov_lin_T1(x1n, y1n, x2n,
y2n, x3n, y3n); Draw_T1(x1, y1, x1n, y1n, x3n, y3n, n - 1); Draw_T1(x2, y2, x1n, y1n, x2n, y2n, n - 1);
Draw_T1(x3, y3, x2n, y2n, x3n, y3n, n - 1); } }
```

5. ПОВНИЙ ЛІСТИНГ ПРОГРАМИ (ФАЙЛ SOURCE.CPP, ЧАСТИНА 2)

```
// ==================== ЗАВДАННЯ 2: КИЛИМ СЕРПІНСЬКОГО ==================== void Prov_lin_T2(double x1, double y1, double x2, double y2) { RectangleFilled((int)round(x1), (int)round(y1), (int)round(x2), (int)round(y2)); } void Draw_T2(double x1, double y1, double x2, double y2, int n) { if (n > 0) { double t = (x2 - x1) / 3.0; Prov_lin_T2(x1 + t, y1 + t, x2 - t, y2 - t); Draw_T2(x1, y1, x1 + t, y1 + t, n - 1); // 1 блок Draw_T2(x1 + t, y1, x1 + 2 * t, y1 + t, n - 1); // 2 блок Draw_T2(x1 + 2 * t, y1, x2, y1 + t, n - 1); // 3 блок Draw_T2(x1, y1 + t, x1 + t, y1 + 2 * t, n - 1); // 4 блок Draw_T2(x1 + 2 * t, y1 + t, x2, y1 + 2 * t, n - 1); // 5 блок Draw_T2(x1, y1 + 2 * t, x1 + t, y2, n - 1); // 6 блок Draw_T2(x1 + t, y1 + 2 * t, x1 + 2 * t, y2, n - 1); // 7 блок Draw_T2(x1 + 2 * t, y1 + 2 * t, x2, y2, n - 1); // 8 блок } } // ==================== ЗАВДАННЯ 3: ДЕРЕВО ПІФАГОРА (ГІЛКИ) ==================== double max_branch = 4.0; void lineto1(int x, int y, double l, double u) { Line(x, y, (int)round(x + l * cos(u)), (int)round(y - l * sin(u))); } void Draw_T3(int x, int y, double l, double u) { if (l > max_branch) { l = l * 0.7; lineto1(x, y, l, u); x = (int)round(x + l * cos(u)); y = (int)round(y - l * sin(u)); Draw_T3(x, y, l, u + PI / 4.0); Draw_T3(x, y, l, u - PI / 6.0); } } // ==================== ЗАВДАННЯ 4: МНОЖИНА КАНТОРА ==================== void Draw_T4(double x1, double x2, int y, int n) { if (n > 0) { RectangleFilled((int)round(x1), y, (int)round(x2), y + 6); double t = (x2 - x1) / 3.0; Draw_T4(x1, x1 + t, y + 25, n - 1); Draw_T4(x2 - t, x2, y + 25, n - 1); } } // ==================== ЗАВДАННЯ 5: ДЕРЕВО ПІФАГОРА (КВАДРАТИ) ==================== void Rect_T5(int x1, int y1, int l, double a1) { MoveTo(x1, y1); LineToBGI(x1 + (int)round(l * cos(a1)), y1 - (int)round(l * sin(a1))); LineToBGI(x1 + (int)round(l * sqrt(2.0) * cos(a1 + PI / 4.0)), y1 - (int)round(l * sqrt(2.0) * sin(a1 + PI / 4.0))); LineToBGI(x1 + (int)round(l * cos(a1 + PI / 2.0)), y1 - (int)round(l * sin(a1 + PI / 2.0))); LineToBGI(x1, y1); } void Draw_T5(double x, double y, double l, double a) { if (l > 4.0) { Rect_T5((int)round(x), (int)round(y), (int)round(l), a); Draw_T5(x - l * sin(a), y - l * cos(a), l / sqrt(2.0), a + PI / 4.0); Draw_T5(x - l * sin(a) + (l / sqrt(2.0)) * cos(a + PI / 4.0), y - l * cos(a) - (l / sqrt(2.0)) * sin(a + PI / 4.0), l / sqrt(2.0), a - PI / 4.0); } } // ==================== ЗАВДАННЯ 6: КРИВА ТА СНІЖИНКА КОХА ==================== void Draw_T6(double x1, double y1, double x2, double y2, int n) { if (n == 0) { Line((int)round(x1), (int)round(y1), (int)round(x2), (int)round(y2)); } else { double xA = x1 + (x2 - x1) / 3.0, yA = y1 + (y2 - y1) / 3.0; double xB = x1 + 2.0 * (x2 - x1) / 3.0, yB = y1 + 2.0 * (y2 - y1) / 3.0; double angle = -PI / 3.0; double xC = xA + (xB - xA) * cos(angle) - (yB - yA) * sin(angle); double yC = yA + (xB - xA) * sin(angle) + (yB - yA) * cos(angle); Draw_T6(x1, y1, xA, yA, n - 1); Draw_T6(xA, yA, xC, yC, n - 1); Draw_T6(xC, yC, xB, yB, n - 1); Draw_T6(xB, yB, x2, y2, n - 1); } } int main() { SetConsoleCP(65001); SetConsoleOutputCP(65001); int choice, n; cout << "=== ЛАБОРАТОРНА РОБОТА №2: ПОБУДОВА ФРАКТАЛІВ ===\n" << "1. Трикутник Серпінського\n2. Килим Серпінського\n3. Дерево Піфагора (гілки)\n" << "4. Множина Кантора\n5. Дерево Піфагора (квадрати)\n6. Крива та сніжинка Коха\nОберіть завдання (1-6): "; cin >> choice; if (choice == 1) { cout << "Порядок (5-7): "; cin >> n; InitWinGraph(); Prov_lin_T1(320, 10, 600, 470, 40, 470); Draw_T1(320, 10, 600, 470, 40, 470, n); } else if (choice == 2) { cout << "Порядок (3-4): "; cin >> n; InitWinGraph(); RectangleOutline(120, 40, 520, 440); Draw_T2(120, 40, 520, 440, n); } else if (choice == 3) { cout << "Мін. довжина гілки (4): "; cin >> max_branch; InitWinGraph(); Draw_T3(320, 460, 300, PI / 2.0); } else if (choice == 4) { cout << "Порядок (5-6): "; cin >> n; InitWinGraph(); Draw_T4(40, 600, 50, n); } else if (choice == 5) { InitWinGraph(); Draw_T5(280, 460, 100, 0); } else if (choice == 6) { cout << "Порядок (4): "; cin >> n; InitWinGraph(); Draw_T6(60, 80, 300, 80, n); double x1 = 360, y1 = 360, x2 = 640, y2 = 360, x3 = 500, y3 = 360 - 280 * sin(PI / 3.0); Draw_T6(x1, y1, x3, y3, n); Draw_T6(x3, y3, x2, y2, n); Draw_T6(x2, y2, x1, y1, n); } else return 0; WaitGraphClose(); return 0; }
```

6. ВИСНОВКИ

У ході виконання лабораторної роботи №2 було досліджено математичні принципи самоподібності та реалізовано алгоритми рекурсивної побудови шести класичних фракталів мовою C++: трикутника та килима Серпінського, гілкового та квадратного дерева Піфагора, множини Кантора, а також кривої та сніжинки Коха. Замість застарілих засобів DOS-графіки BGI розроблено власний модуль ініціалізації графічного вікна на базі Win32 API (GDI), що забезпечило стабільну роботу програми у середовищі Windows 11 та коректне відображення україномовного інтерфейсу в кодуванні UTF-8.